Kindle サイネージ 構築手順書

項目	内容
文書名	Kindle サイネージ 構築手順書
対象システム	kindle_dashboard（J105588/kindle_dashboard）
対象機器	Kindle 第7世代（Paperwhite 第7世代＝PW3 / Voyage / 無印 第7世代＝KT2）
作業用PC	Windows 10/11（PowerShell）
最終更新	2026-10-12
状態	1〜10章は実機で稼働確認済み。11章（Google Drive 同期）は実装済み・実機未検証

目次

0. 本書について
1. システム概要
2. 必要な機材・ソフトウェアと配布元
3. 手順1 脱獄の確認と Hotfix の適用
4. 手順2 KUAL・MRPI の導入
5. 手順3 PC 側サーバーの構築
6. 手順4 Kindle 側サイネージ拡張の配置
7. 手順5 サイネージの運用
8. 手順6 スクリーンセーバーの設定
9. 手順7 画像の変換
10. 手順8 SSH（USBNetwork）の導入
11. 手順9 SSH による運用（画像送信・更新）
12. 手順10 PC なし運用（GitHub Actions ＋ GitHub Pages）
13. 手順11 Google Drive 同期と KOReader
14. 付録A 設定項目一覧
15. 付録B ファイル配置
16. 付録C コマンド一覧
17. 付録D 障害対応
18. 付録E 撤去手順

19. 付録F 安全上の注意
20. 付録G 参考資料
21. 変更履歴

---

0. 本書について

0-1. 目的

本書は、脱獄した Kindle 第7世代を次の用途に使うための構築手順をまとめたものである。

- 時計・日付・天気・カレンダーを常時表示するサイネージ
- スリープ画面を任意の画像に差し替えるスクリーンセーバー
- スマホ・PC から Wi-Fi 経由で画像を送り、プログラムを更新する仕組み（SSH）
- PC を使わずに表示内容を更新する仕組み（GitHub Actions）
- Google Drive の PDF を KOReader で読む仕組み（rclone）

いずれも Kindle 本体のシステム領域は書き換えず、読書端末としての機能を保ったまま追加する。

0-2. 表記

- `E:` は、Kindle を USB 接続したときのドライブ文字を表す。環境に合わせて読み替える。
- `C:\dev\tmp\kindle_dashboard` は、PC 上の作業フォルダである。
- `/mnt/us` は Kindle 内部のパスで、PC から見た Kindle ドライブの直下（`E:\`）にあたる。
- 本書の IP アドレス（PC `192.168.1.134`、Kindle `192.168.1.177`）は 2026-10-12 時点の構築環境の値である。
- Kindle に置くテキストファイル（`.sh`、`config.sh`、`authorized_keys`）は、**改行コード LF** で保存する。CRLF では動作しない。

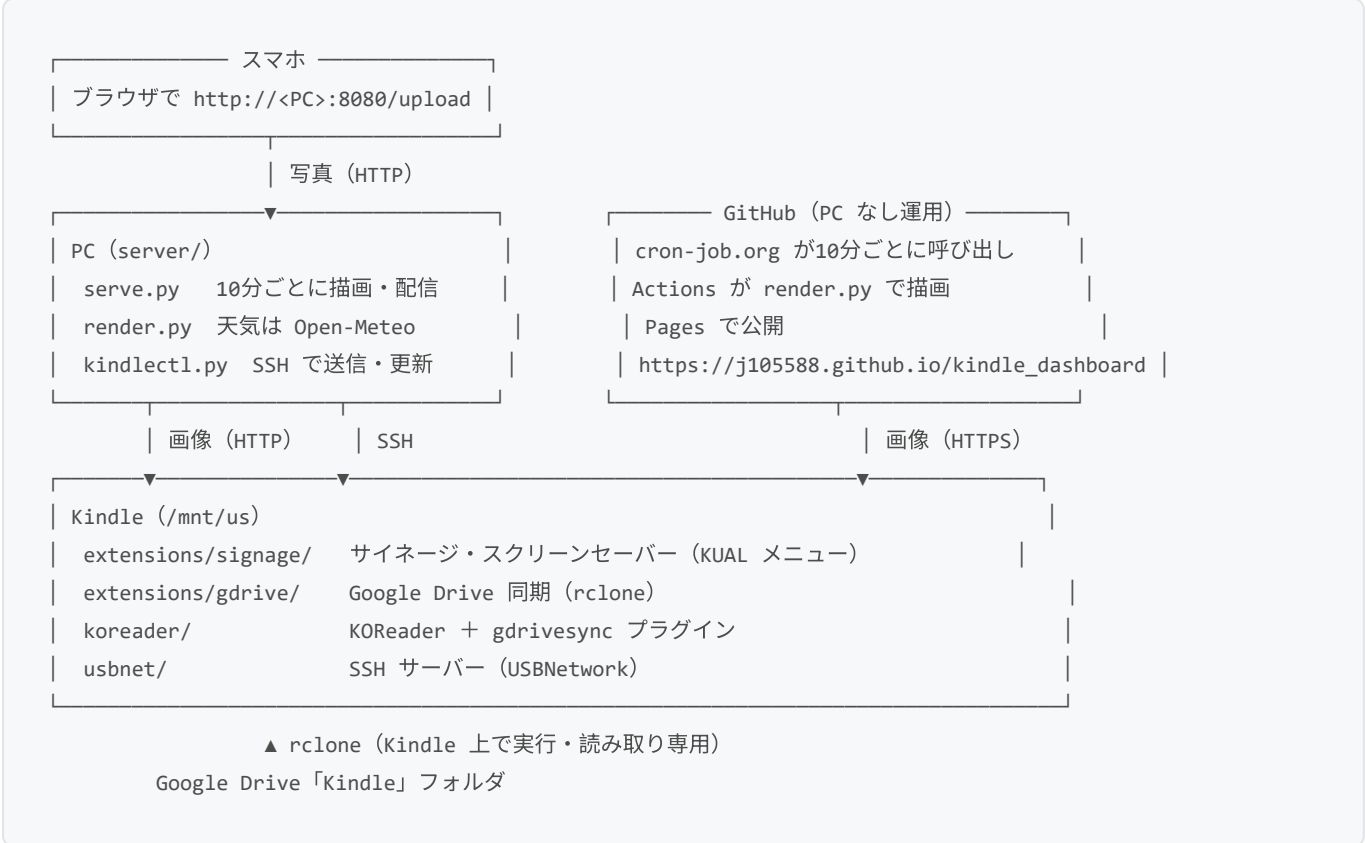
0-3. 作業の流れと所要時間の目安

手順	内容	目安
1	脱獄の確認・Hotfix	10分
2	KUAL・MRPI	10分
3	PC 側サーバー	20分
4	Kindle 側配置	10分
5〜7	サイネージ・スクリーンセーバー・画像	20分
8〜9	SSH の導入と運用	30分
10	PC なし運用（任意）	30分
11	Google Drive 同期（任意）	30分

---

1. システム概要

1-1. 構成



1-2. 構成要素

要素	置き場所	役割
render.py	PC `server\` / GitHub Actions	ダッシュボード画像（背景・時計の数字）を描く
serve.py	PC `server\`	描画を定期実行し、HTTP で配信する。スマホ用アップロード画面も持つ
kindlectl.py	PC `server\`	SSH で Kindle を操作する（画像送信・設定変更・更新・Drive 同期の設定）
signage 拡張	Kindle `extensions/signage/`	サイネージ表示、スクリーンセーバー、タッチ独占（touchgrab）
gdrive 拡張	Kindle `extensions/gdrive/`	rclone による Drive → Kindle の一方向同期
gdrivesync プラグイン	Kindle `koreader/plugins/`	KOReader のメニューから同期を操作する
USBNetwork	Kindle `usbnet/`	SSH サーバー

1-3. 設計上の要点

- **描画は PC（または GitHub）、Kindle は表示のみ。** Kindle には日本語の描画環境がないため。
- **時計は Kindle が進める。** 数字0〜9の画像を先に受け取り、変わった桁だけ差し替える。通信は10分に1回で済み、画面も点滅しない。
- **背景は Kindle 時計の 09・19・29…分に取り直す**（`DASH_REFRESH_OFFSET=9`）。
- **天気は毎時ちょうどを避けて取得する。** 天気サービスが毎時0分に混雑し、HTTP 503 を返すため。
- **本体のシステム領域を書き換えない。** すべて `/mnt/us` 内で完結し、再起動で常駐処理はすべて止まる。

2. 必要な機材・ソフトウェアと配布元

2-1. 機材

機材	条件・備考
Kindle 第7世代	脱獄可能なファームウェアであること。機種とファームウェアは **設定 → 端末オプション → 端末情報** で確認する
micro-USB ケーブル	データ通信対応のもの（充電専用は不可）
Windows PC	Windows 10/11。PowerShell と `ssh` コマンドが標準で入っている
Wi-Fi	**2.4GHz** の SSID が必要（第7世代は 5GHz に接続できない）。PC と Kindle は同じネットワークにつなぐ
USB 電源	常時表示する場合は給電したまま運用する

2-2. ソフトウェアと配布元

名称	版	配布元	使う手順
Kindle Modding Guide（脱獄手順）	—	https://kindlemodding.org/ （WinterBreak: https://kindlemodding.org/jailbreaking/WinterBreak/ ）	1
Universal Hotfix（`Update_hotfix_universal.bin`）	v1.8.0〜v2.3.7 を推奨（下記注意）	https://github.com/KindleModding/Hotfix/releases	1
MRPI（MobileRead Package Installer）	modern devices 用	Kindle Modding Guide「Installing KUAL and MRPI」 https://kindlemodding.org/jailbreaking/Legacy/post-jailbreak/installing-kual-mrpi/	2
PEKI（KUAL のインストーラー兼ランチャー）	最新	https://github.com/KindleTweaks/PEKI/releases	2
Python	3.9 以上	https://www.python.org/downloads/windows/	3
Git for Windows（任意）	最新	https://git-scm.com/download/win	3
7-Zip	最新	https://www.7-zip.org/	8
kindle_dashboard（本システム）	main	https://github.com/J105588/kindle_dashboard	3 以降
USBNetwork	0.22.N r19297	配布スレッド: https://www.mobileread.com/forums/showthread.php?t=225030 （「Snapshots of NiLuJe's hacks」の最初の投稿） ファイル: https://storage.gra.cloud.ovh.net/v1/AUTH_2ac4bfee353948ec8ea7fd1710574097/mr-public/Touch/kindle-usbnet-0.22.N-r19297.tar.xz	8

名称	版	配布元	使う手順
KOReader	最新（例 v2026.07.2）	https://github.com/koreader/koreader/releases （導入手順: https://kindlemodding.org/jailbreaking/post-jailbreak/koreader.html ）	11
rclone	**v1.69.0 に固定**	https://github.com/rclone/rclone/releases/tag/v1.69.0 （`kindlectl.py gdrive-setup` が自動で取得する）	11
Open-Meteo（天気 API）	—	https://open-meteo.com/ （API キー不要。プログラムが自動で使う）	3
cron-job.org	—	https://cron-job.org/	10

Hotfix と PEKI の版についての注意

- Hotfix は v1.8.0 より前の版に、文鎮化の恐れがある不具合が報告されている。v1.8.0 以降を使う。
- PEKI の README には「Universal Hotfix 2.3.7 以下のみ対応」と書かれている。KUAL を使う本システムでは、Hotfix は **v1.8.0〜v2.3.7** の範囲を使う。2026年10月時点で Hotfix の最新タグは v3.4.0 だが、新しい系統は KUAL を前提としていない。
- PEKI はアーカイブ（開発終了）済みで、ファームウェア 5.19.4 以降では動作しない。第7世代の最終ファームウェアはこれより古いため、影響はない。

KOReader のパッケージの選び方

ファイル名は `koreader-<種類>-<版>.zip` である。

種類	対象
`kindlepw2`	Paperwhite 第2世代以降で、ファームウェアが 5.16.3 より前の機種。**第7世代は通常これ**
`kindlehf`	ファームウェア 5.16.3 以降
`kindle` / `kindle-legacy`	さらに古い機種

---

3. 手順1 脱獄の確認と Hotfix の適用

目的: 脱獄状態を確認し、OTA アップデート後も脱獄を保てるようにする。

前提: Kindle Modding Guide の手順（WinterBreak など）で脱獄済みであること。

3-1. 脱獄の確認

1. Kindle のライブラリを開く。
2. **JAILBROKEN** という本があることを確認する。

3-2. Hotfix の入手

1. https://github.com/KindleModding/Hotfix/releases を開く。
2. v1.8.0〜v2.3.7 のいずれかのリリースの **Assets** から `Update_hotfix_universal.bin` を保存する。
3. ファイル名に `(1)` などが付いた場合は消す。

3-3. Hotfix の適用

第7世代では「Kindle をアップデート」がグレーアウトして押せないことがある。そのため、手順2で入れる MRPI を使って適用する（手順2の 4-2 で `mrpackages` に入れる）。

「Kindle をアップデート」が押せる場合は、次の方法でもよい。

1. `Update_hotfix_universal.bin` を Kindle ドライブの直下（`documents` と同じ階層）に置く。
2. 安全に取り外す。
3. **設定 → 右上の「⋮」→ Kindle をアップデート** を押す。

3-4. Run Hotfix の実行

1. 適用後、ライブラリに **Run Hotfix**（または Run Bridge）が現れる。
2. 一度タップして実行する。
3. OTA アップデートが入ったときは、再度実行する。

3-5. 自動アップデートの抑止

1. Kindle ドライブの直下に、`update.bin.tmp.partial` という名前の**空フォルダ**を作る（定番の対策だが確実ではない）。
2. Wi-Fi を使わない作業中は機内モードを ON にしておく。

確認: ライブラリに JAILBROKEN と Run Hotfix がある。

---

4. 手順2 KUAL・MRPI の導入

目的: 自作メニューを動かす KUAL と、パッケージをインストールする MRPI を入れる。

前提: 手順1が完了していること。第7世代は「K5 以降」に該当するため、KUAL は PEKI を使う。

4-1. 空き容量の確保

1. **機内モードを ON** にする（容量を空けた瞬間に OTA が入るのを防ぐ）。
2. 空き容量が **220MB 以上**あることを確認する。不足していれば本を削除する。

4-2. ファイルの配置

1. MRPI（modern devices 用）と PEKI（`PEKI.zip`）をダウンロードする。配布元は 2-2 の表のとおり。
2. Kindle を PC に USB 接続する。
3. MRPI を展開し、`extensions` と `mrpackages` を `E:\` にコピーする。
4. Hotfix の `.bin` を `E:\mrpackages\` に入れる。
5. `PEKI.zip` を展開し、`KUAL.sh` と `KUAL.jar` を `E:\documents\` にコピーする。
6. 「ハードウェアを安全に取り外す」を行ってからケーブルを抜く。

4-3. インストール

1. ホームの検索バーに `;log mrpi` と入力して確定する。
2. ログが表示され、再起動すれば Hotfix の適用が完了する。
3. ライブラリの **KUAL** を一度開く（PEKI が KUAL を設定する）。
4. **Run Hotfix** があれば一度実行する。

確認: KUAL を開くとメニューが表示される。

異常時:

- `;log mrpi` が反応しない場合は、Kindle を再起動して再度実行する。
- KUAL を開くと真っ白な場合は、KUAL の版が機種に合っていない。PEKI を入れ直す。

---

5. 手順3 PC 側サーバーの構築

目的: ダッシュボード画像を描いて配信する PC 側の環境を作る。

5-1. Python のインストール

1. https://www.python.org/downloads/windows/ から Python 3.9 以上をダウンロードする。
2. インストール時に **Add python.exe to PATH** にチェックを入れる。
3. PowerShell で `python --version` を実行し、版が表示されることを確認する。

5-2. 本システムの取得

Git を使う場合:

```
git clone https://github.com/J105588/kindle_dashboard.git C:\dev\tmp\kindle_dashboard
```

Git を使わない場合は、GitHub のリポジトリ画面の **Code → Download ZIP** で取得し、`C:\dev\tmp\kindle_dashboard` に展開する。

5-3. 依存パッケージのインストール

`requirements.txt` は `server` フォルダの中にあるため、先に移動する。

```
cd C:\dev\tmp\kindle_dashboard\server
pip install -r requirements.txt
```

インストールされるもの: Pillow（描画）、jpholiday（祝日）、tzdata（タイムゾーン）、certifi（証明書。手順11で使う）。

5-4. 設定

`server\config.json` を編集する。項目は付録A-1 のとおり。最低限、次の項目を確認する。

- `latitude` / `longitude`: 天気を取得する地点（Google マップで地点を右クリックすると表示される）
- `device`: 機種（`pw3` / `voyage` / `kt2`）

5-5. 表示の確認

```
python render.py --demo
```

- `out\preview.png` が作られる（ダミーの天気、時刻入り）。開いて表示を確認する。
- `out\dashboard.png` は Kindle が取得する背景で、時計の数字は空白になっている（Kindle が自分で描くため）。

5-6. サーバーの起動

1. 前面で起動して確認する。

   ```
   python serve.py
   ```

2. `配信中: http://192.168.1.134:8080` のように表示された URL を控える。
3. Windows ファイアウォールの確認が出たら、**プライベートネットワーク**を許可する。
4. ブラウザで `http://192.168.1.134:8080/preview.png` が表示されることを確認する。
5. `Ctrl+C` で止め、バックグラウンド起動に切り替える。

   ```
   python serve.py start
   python serve.py autostart on
   ```

コマンド	内容
`python serve.py start`	バックグラウンドで起動する（PowerShell を閉じても動き続ける）
`python serve.py stop`	停止する
`python serve.py restart`	再起動する（`config.json` や `.py` を変更したとき）
`python serve.py status`	稼働状況・URL・自動起動の状態を表示する
`python serve.py log`	ログ（`out\serve.log`）の末尾を表示する
`python serve.py autostart on` / `off`	Windows サインイン時の自動起動を設定・解除する

5-7. ログの見方

```
[00:22:26] 描画しました
[00:23:06] Kindle からアクセス: 192.168.1.177  ← これが Kindle の IP アドレス
[00:30:02] 描画しました
[00:37:52] 天気を更新しました
```

- 画面は10分ごと（0・10・20…分）に描き直す。
- 天気は30分ごとに、毎時7分・37分ごろ取得する。
- 天気の取得に失敗したときは前回のデータで描く。その場合、画面の「3時間ごと」の横に「(天気は 15:20 時点)」と表示される。

確認: `python serve.py status` で稼働中と表示され、`preview.png` がブラウザで開ける。

---

6. 手順4 Kindle 側サイネージ拡張の配置

目的: サイネージとスクリーンセーバーの拡張を Kindle に入れる。

6-1. コピー

1. Kindle を USB 接続する。
2. 旧版が入っている場合は、`E:\extensions\signage` だけを削除する。`E:\extensions` 自体と、`usbnet`・`mrpackages` は削除しない。
3. `C:\dev\tmp\kindle_dashboard\kindle\` の中の `extensions` と `signage` を `E:\` にコピーし、「統合」を選ぶ。

   ```
   Copy-Item -Recurse -Force C:\dev\tmp\kindle_dashboard\kindle\extensions\signage E:\extensions\
   Copy-Item -Recurse -Force C:\dev\tmp\kindle_dashboard\kindle\signage E:\
   ```

6-2. 接続先の設定

1. `E:\extensions\signage\config.sh` を開く。

2. `SERVER_URL` を手順3で控えた URL にする。

```
SERVER_URL="http://192.168.1.134:8080"
```

3. **LF で保存する。** メモ帳の場合は、右下の「CRLF」を「LF」に切り替えてから保存する。

SSH 導入後は `python kindlectl.py set SERVER_URL ...` で変更できる（手順9）。

6-3. 確認

1. Kindle を安全に取り外し、Wi-Fi を ON（機内モード OFF）にする。
2. KUAL を開き、**Signage** メニューがあることを確認する。
3. **Signage → Status** を押す。画面下に次のように表示されれば完了である。

```
Kindle IP: 192.168.1.177
Signage: stopped (dashboard)
Screensaver: OFF
Server: OK  http://192.168.1.134:8080
Clock now: 10:42
```

異常時: `Server: unreachable` のときは、PC と Kindle が同じ Wi-Fi にいるか、ファイアウォールの許可、`SERVER_URL` を確認する。

---

7. 手順5 サイネージの運用

7-1. KUAL メニュー

メニュー	動作
Signage: Start	サイネージを開始する（数秒後に表示）
Signage: Stop	サイネージを終了し、ホーム画面に戻る
Screensaver: ON / OFF	スリープ画面の差し替えを切り替える
Screensaver: Preview	スリープ画面の画像を10秒間表示する
Status	Kindle の IP、稼働状況、サーバー接続、時刻を表示する

7-2. 開始と終了

1. PC のサーバーが稼働していることを確認する（PC なし運用では不要）。
2. KUAL → **Signage: Start** を押す。
3. 終了する方法は次の2つである。
   - **2秒以内に画面を3回タップ**すると、ホーム画面に戻る。読書に戻るときはこちらを使う。
   - **電源ボタン**を押すと、終了してそのままスリープする。
4. 再開するときは **Signage: Start** を押す。

7-3. 動作仕様

項目	動作
時計	Kindle が毎分、変わった桁だけを書き換える
背景	10分ごと（09・19・29…分）に取得し、全画面を書き換える
時刻合わせ	1時間ごとに `ntp.nict.jp` で補正する
電池	残量が5%以下（充電中を除く）になると、右上に電池マークを表示する
タッチ	サイネージがタッチパネルを独占し、裏のホーム画面や本にタップが届かない
裏の画面	タッチを独占できたときだけ、Kindle 本来の画面を一時停止する。独占に失敗した場合は安全のため停止しない（ログに `safe mode` と出る）
通信障害	前回の画像のまま時計だけ進める。3回以上失敗すると左上に `offline` と表示する

7-4. スライドショーへの切り替え

`config.sh` を `SIGNAGE_SOURCE=slides` にすると、`E:\signage\slides` の画像を60秒ごと（`SLIDE_INTERVAL`）に順番に表示する。PC も Wi-Fi も不要である。

7-5. 画面が固まった場合

電源ボタンを**約40秒長押し**して再起動する。再起動すると、画面の一時停止とタッチの独占は必ず解除される。本などのデータは消えない。

8. 手順6 スクリーンセーバーの設定

目的: スリープ画面を任意の画像に差し替える。読書中の動作には影響しない。

8-1. 画像の用意

次のいずれかの方法で、`signage/screensaver` に PNG を入れる。

- スマホのアップロード画面で、送り先に「スクリーンセーバー」を選ぶ（手順9）
- PC から `python kindlectl.py push 写真フォルダ` を実行する（手順9）
- USB 接続して `python convert_images.py 写真フォルダ --kindle E:/ --to screensaver` を実行する（手順7）

8-2. 有効化

1. KUAL → **Screensaver: Preview** で画像を確認する。
2. **Screensaver: ON** を押す。画面下に `Screensaver: ON (native)` と出れば最適な状態である。
3. 電源ボタンでスリープさせて確認する。
4. 無効にするときは **Screensaver: OFF** を押す。

Kindle を再起動すると OFF に戻る。 再起動後は ON を押し直す。

8-3. 表示方式（`SS_METHOD`）

方式	仕組み	見え方
`auto`（既定）→ native	Kindle の標準スリープ画像の場所に、RAM 上で自分の画像を重ねる。Kindle 自身がその画像を描く	自分の画像だけが出る。ちらつかない
`draw`（保険）	標準画像が描き終わったのを確認してから、上から1回描く	標準画像の後に1回切り替わる

`auto` の場合でも、native が使えない端末（広告付きなど）や `SS_SOURCE=dashboard` のときは自動で `draw` になる。どちらで動いているかは、`python kindlectl.py log -n 5` の `screensaver ON (native, images)` の部分で確認できる。

9. 手順7 画像の変換

`convert_images.py` は、写真を Kindle の解像度に合わせて切り抜き、E-ink 向けに補正して16階調のグレーにする。スマホ送信や `kindlectl.py push` では自動で同じ変換がかかるため、このコマンドは USB で入れるときに使う。

```
cd C:\dev\tmp\kindle_dashboard\server
python convert_images.py "C:\Users\<ユーザー名>\Pictures\写真"
python convert_images.py "C:\Users\<ユーザー名>\Pictures\写真" --kindle E:/ --to screensaver
python convert_images.py "C:\Users\<ユーザー名>\Pictures\写真" --kindle E:/ --to slides --fit contain --bg
black --rotate auto
```

オプション	内容
`--fit cover` / `contain`	画面いっぱいに切り抜く（既定）/ 全体を収めて余白を付ける
`--bg white` / `black`	`contain` の余白の色
`--rotate auto`	横長の画像を90° 回転して大きく表示する
`--no-dither`	イラスト・文字・ロゴ向け（点描を使わない）
`--contrast` / `--brightness` / `--sharpen`	見た目の調整（既定 1.15 / 1.05 / 1.0）
`--device`	解像度。既定は `config.json` の `device`（PW3・Voyage は 1072×1448、KT2 は 600×800）

対応形式は JPG / PNG / WebP / BMP / GIF / TIFF である。HEIC は `pip install pillow-heif` を追加すると扱える。

---

10. 手順8 SSH（USBNetwork）の導入

目的: USB ケーブルなしで Kindle を操作できるようにする。

重要: 鍵の登録 → 鍵での接続確認 → Wi-Fi 経由の有効化の順を必ず守る。Wi-Fi 経由の SSH や OpenSSH を有効にすると、パスワードなしのログインが無効になる。鍵の登録前に有効にすると、ログインできなくなる（MobileRead に事例がある）。万一締め出された場合も、USB でファイルを直せば復旧できる。

10-1. USBNetwork のインストール

1. 次のファイルをダウンロードする。
   https://storage.gra.cloud.ovh.net/v1/AUTH_2ac4bfee353948ec8ea7fd1710574097/mr-public/Touch/kindle-usbnet-0.22.N-r19297.tar.xz （最新版は https://www.mobileread.com/forums/showthread.php?t=225030 の最初の投稿で確認する）
2. 7-Zip で2回展開する（`.tar.xz` → `.tar` → 中身）。`.tar.xz` をそのまま Kindle に入れても動作しない。
3. 中の `Update_usbnet_0.22.N_install_pw2_and_up.bin` を `E:\mrpackages\` に入れる。`uninstall` や `touch_pw` のファイルは使わない。
4. 安全に取り外し、検索バーで `;log mrpi` を実行する。
5. 再度 USB 接続し、`E:\usbnet` フォルダができていることを確認する。

10-2. 鍵の作成と登録（USB 接続のまま）

```
cd C:\dev\tmp\kindle_dashboard\server
powershell -ExecutionPolicy Bypass -File .\setup_ssh.ps1 -Drive E:
```

- 鍵（`C:\Users\<ユーザー名>\.ssh\kindle` と `kindle.pub`）を作成する。
- 公開鍵を `E:\usbnet\etc\authorized_keys` に、拡張子なし・LF・BOM なしで書き込む。
- 最後に `改行 LF（OK）` と表示されれば成功である。

手作業で作る場合は、`authorized_keys` を**フォルダではなく拡張子なしのファイル**にする。Kindle の直下に `.ssh` を作っても効果はない。

10-3. Wi-Fi 経由の SSH の有効化

1. Kindle を安全に取り外す。
2. KUAL → USBNetwork メニューで、Wi-Fi 経由の SSH を許可する項目（「Allow SSH over WiFi」など）を選ぶ。項目名は版によって異なるため、同梱の `README_FIRST.txt` で確認する。
3. 同じメニューで **USBNetwork を ON** にする（検索バーの `;un` でも切り替えられる）。
4. SSH の種類（OpenSSH / Dropbear）は変更しない。

USBNetwork が ON の間は、PC に接続しても Kindle がドライブとして表示されないことがある。USB でファイルをコピーするときは OFF にする。

10-4. IP アドレスの確認

次のいずれかで確認する。

- KUAL → Signage → **Status** の `Kindle IP:` の行
- `python serve.py log` の「Kindle からアクセス: …」の行
- ルーターの管理画面の接続機器一覧

ルーターの DHCP 予約で、Kindle に固定の IP を割り当てておくとよい。

10-5. 接続テスト

Kindle の Wi-Fi を ON にし、スリープさせない状態で実行する。

```
ssh -i $HOME\.ssh\kindle root@192.168.1.177
```

- 初回は `Are you sure you want to continue connecting?` に `yes` と答える。
- `Welcome to Kindle!` の後に入力待ちになれば成功である。`exit` で抜ける。
- パスワードを聞かれた場合は、何も入力せず Ctrl+C で中断する。USBNetwork を OFF にして USB 接続し、10-2 をやり直す。

10-6. ツールの接続設定

`server\config.json` の `kindle` を設定する。

```
"kindle": { "host": "192.168.1.177", "user": "root", "port": 22, "identity": "~/.ssh/kindle" }
```

```
python kindlectl.py check
```

確認: `接続OK` と表示される。

---

11. 手順9 SSH による運用（画像送信・更新）

注意: Kindle がスリープ中は Wi-Fi が切れるため、送信できない。サイネージ表示中は送信できる。

11-1. スマホからの画像送信

1. PC のサーバーが稼働していることを確認する（`python serve.py status`）。
2. スマホのブラウザで `http://192.168.1.134:8080/upload` を開く。ホーム画面に追加しておくと便利である。
3. 写真を選び、送り先（スクリーンセーバー / スライドショー）と収め方を選んで **Kindle に送る** を押す。
4. 画面上部に「〇枚を…に送りました」と表示されれば完了である。

同じ Wi-Fi の端末なら誰でも送信できる。制限する場合は `config.json` に `"upload_pin": "1234"` のように PIN を設定する。

11-2. PC からの操作

コマンドの一覧は付録C を参照する。主なものは次のとおり。

```
python kindlectl.py push 写真フォルダ               # スクリーンセーバー用に送る
python kindlectl.py push a.jpg --to slides --show   # スライド用に送り、すぐ表示
python kindlectl.py set SS_METHOD auto              # Kindle の設定を変える
python kindlectl.py log -n 20                       # Kindle のログを見る
```

11-3. プログラムの更新

PC 側の `kindle\` を新しい版にしてから（`git pull` など）、次を実行する。

```
python kindlectl.py update
```

更新は次の順で自動的に行われる。

1. 稼働中のサイネージとスクリーンセーバーを停止する。
2. 現在の版を Kindle 内の `signage/.backup/` に保存する（最新5つを保持。rclone 本体は含めない）。
3. 新しい版を送る。**各 `config.sh` は上書きしない。** 改行は自動で LF に直す。Drive 同期を導入済みの Kindle には、gdrive 拡張と KOReader プラグインも送る。

4. 停止したものを元どおりに起動する。

コマンド	内容
`python kindlectl.py update --dry-run`	送るファイルの確認のみ
`python kindlectl.py update --with-config`	`config.sh` も上書きする
`python kindlectl.py rollback`	直前の更新を取り消す

新しい版で設定項目が増えても、`config.sh` を上書きせずに既定値で動作する。変更したい値だけを `set` で変える。

---

12. 手順10 PC なし運用（GitHub Actions ＋ GitHub Pages）

目的: PC の電源が切れていても、ダッシュボードを更新し続ける。

構成: cron-job.org が10分ごとに GitHub Actions を呼び出す。Actions が `render.py --refresh-weather` で描画し、GitHub Pages で公開する。Kindle はインターネット経由で取得する。スマホ送信と `kindlectl.py` は、引き続き同じ Wi-Fi 内の PC から使う。

12-1. GitHub Pages の有効化

1. リポジトリ https://github.com/J105588/kindle_dashboard を **Public** にする（無料で使うための条件）。`server\config.json` の地点と Kindle の IP も公開されるため、必要に応じて書き換える。
2. **Settings → Pages → Build and deployment → Source** を **GitHub Actions** にする。
3. `git push` 後、**Actions → Dashboard → Run workflow** で1回手動実行する。
4. https://j105588.github.io/kindle_dashboard/preview.png が表示されることを確認する。

12-2. 外部からの定期呼び出し（cron-job.org）

GitHub 自身の定期実行（`schedule`）は数十分〜数時間遅れることがあるため、毎時35分の保険としてのみ使う。

1. **トークンの作成** GitHub 右上のアイコン → **Settings → Developer settings → Personal access tokens → Fine-grained tokens → Generate new token**

項目	設定
Token name	`kindle-dashboard-cron` など
Expiration	1年など（期限が来たら作り直し、cron-job.org の設定を差し替える）
Repository access	Only select repositories → `kindle_dashboard` のみ
Permissions	**Actions: Read and write** のみ

   表示された `github_pat_...` を控える（一度しか表示されない）。

2. **PC からの試験呼び出し**

```
curl.exe -sf -X POST -H "Authorization: Bearer <トークン>" -H "Accept: application/vnd.github+json" `
  https://api.github.com/repos/J105588/kindle_dashboard/actions/workflows/dashboard.yml/dispatches -d
'{\"ref\":\"main\"}'
```

   何も表示されずに終わり、Actions 画面に `workflow_dispatch` の実行が1件増えれば成功である。

3. **cron-job.org への登録**（https://cron-job.org/ 、無料・要アカウント）。**CREATE CRONJOB** で次のように設定する。

項目	設定
URL	`https://api.github.com/repos/J105588/kindle_dashboard/actions/workflows/dashboard.yml/dispatches`
Execution schedule	Custom → Minutes に `5,15,25,35,45,55`、ほかは Every
Time zone	Asia/Tokyo
Request method	POST
Headers	`Authorization: Bearer <トークン>`、`Accept: application/vnd.github+json`、`X-GitHub-Api-Version: 2022-11-28`、`Content-Type: application/json`
Request body	`{"ref":"main"}`
Notifications	失敗時のメール通知を ON（トークン切れに気づけるようにする）

   保存後、**TEST RUN** で `204` が返れば完了である。

12-3. Kindle の接続先の切り替え

```
python kindlectl.py set SERVER_URL https://j105588.github.io/kindle_dashboard
```

確認: KUAL → Signage → Status で `Server: OK` と表示される。

PC 版に戻すときは、`SERVER_URL` を `http://192.168.1.134:8080` に戻す。

12-4. 運用上の注意

- 地点などを変えるときは、`server\config.json` を編集して push する。
- 公開リポジトリは、60日間更新がないと定期実行が停止される。毎週月曜にワークフローが自身を有効化し直すが、停止した場合は Actions 画面の **Enable workflow** で再開する。
- トークンで可能な操作は、このリポジトリの Actions の操作のみである。漏洩した場合は GitHub で直ちに Revoke する。
- 古いファームウェアの Kindle は HTTPS の証明書を検証できないことがあるため、自動で検証なしの取得に切り替える。

---

13. 手順11 Google Drive 同期と KOReader

状態: 実装済み・実機未検証（2026-10-12 時点）。

目的: 任意の端末から Google Drive の「Kindle」フォルダに PDF を入れると、Kindle の `/mnt/us/gdrive` に同期され、KOReader で読めるようにする。

13-1. 仕様

Drive での操作	Kindle での結果
PDF を追加	次の同期で Kindle に入る（サブフォルダも再現する）
PDF を上書き	Kindle のファイルも更新される。KOReader の読書位置（`*.sdr`）は保持する
PDF を削除	Kindle からも削除される。`/mnt/us/gdrive_trash` に14日間保管する

- 同期は Drive → Kindle の**一方向**である。Kindle 側で削除しても Drive からは消えない。

- Google の権限は**読み取り専用**（`drive.readonly`）である。Kindle から Drive を変更することはない。
- 1回の同期で削除されるファイルが `MAX_DELETE`（既定20）を超える場合は、**1件も削除せずに中止**する。Drive のフォルダ名を誤った場合も削除しない。
- 同期のタイミングは、**電源ボタンを押して画面がついたとき（スリープからの復帰時、自動）と手動**（KOReader / KUAL のメニュー、PC の `gdrive-sync`）のみである。
  - 復帰時の同期は、Wi-Fi に45秒以内につながらなければ何もせず終了する。前回の同期から3分以内（`MIN_GAP_MIN`）の復帰では同期しない。
  - KOReader で読書中は、変化があった場合と失敗した場合だけ結果を表示する。Kindle 標準の画面での復帰は、常駐の待ち受け（`wakesync.sh`）が同期する。
  - Kindle を再起動すると待ち受けは止まる。次に KOReader を起動したとき、または KUAL の **GDrive: Sync on power button ON** で再開する。
- rclone は **v1.69.0 に固定**している。v1.69.1 以降は Go 1.24 でビルドされており Linux 3.2 以上を要求するため、第7世代のカーネル（3.0系）では起動しない可能性がある。

13-2. 前提

1. Google Drive の**マイドライブ直下に「Kindle」フォルダ**を作る（別名にする場合は `--folder 名前`）。
2. KOReader を導入する。
   1. https://github.com/koreader/koreader/releases から `koreader-kindlepw2-<版>.zip` をダウンロードする（ファームウェア 5.16.3 以降の機種は `kindlehf`）。
   2. 展開した中の `koreader` と `extensions` を `E:\` にコピーする（統合する）。
   3. KUAL → **KOReader** から起動できることを確認する。
   4. 詳細は https://kindlemodding.org/jailbreaking/post-jailbreak/koreader.html を参照する。
3. 手順8（SSH）が完了していること。
4. PC で `pip install -r requirements.txt` を再実行する（`certifi` を追加している）。

13-3. セットアップ（PC で1回のみ）

Kindle の Wi-Fi を ON にし、画面をつけた状態で実行する。

```
cd C:\dev\tmp\kindle_dashboard\server
python kindlectl.py gdrive-setup
```

処理内容:

1. rclone v1.69.0（PC 用・Kindle 用）と証明書を `server\gdrive\` にダウンロードする。このフォルダは `.gitignore` で除外している。

2. **ブラウザが開くので、Google にログインして「許可」を押す。**
3. Drive の「Kindle」フォルダの存在と、PDF の件数を確認する。
4. Kindle に gdrive 拡張、KOReader プラグイン、rclone（約60MB）、証明書、ログイン情報を送る。
5. 初回の同期を行う。

オプション	内容
`--folder 名前`	同期する Drive のフォルダ（既定 `Kindle`）
`--reauth`	Google へのログインをやり直す
`--client-id ID --client-secret SECRET`	自分の OAuth クライアントでログインする（13-4）
`--with-config`	Kindle の `gdrive/config.sh` も上書きする
`--no-sync`	転送のみ行い、同期しない

確認:

```
python kindlectl.py gdrive-status
python kindlectl.py gdrive-log -n 30
```

`前回の同期: 成功` と表示され、ログに rclone の `Copied (new)` などが出ていればよい。初回は、rclone が Kindle で起動しているかを必ずログで確認する。

13-4. 自分の OAuth クライアント ID を使う（推奨）

rclone の既定のクライアント ID は**世界中の rclone 利用者で共有**しており、Drive API の上限に頻繁に当たる。ログに次のようなエラーが出て同期できない場合は、自分のクライアント ID を作って使う（無料）。

```
CRITICAL: Failed to create file system for "kgdrive:Kindle": couldn't find root directory ID:
googleapi: Error 403: Quota exceeded for quota metric 'Queries' ... rateLimitExceeded
```

1. https://console.cloud.google.com/ で新しいプロジェクトを作る（名前は任意。例 `kindle-gdrive`）。
2. **API とサービス → ライブラリ** で「Google Drive API」を検索し、**有効にする**。
3. **API とサービス → OAuth 同意画面（Google Auth Platform）** を開き、次のとおり設定する。
   - User Type: **外部**。アプリ名・サポートメール・連絡先は自分のものでよい。
   - **データアクセス（スコープ）** に `.../auth/drive.readonly` を追加する。
   - **対象（テストユーザー）** に Kindle で使う Google アカウントを追加する。
   - **公開ステータスを「本番環境」にする**（「テスト」のままだとログインが7日で切れる。確認審査は不要で、ログイン時に「Google はこのアプリを確認していません」と出たら **詳細 → 移動** で進める）。

4. **API とサービス → 認証情報 → 認証情報を作成 → OAuth クライアント ID**、アプリケーションの種類は **デスクトップ アプリ** で作成し、クライアント ID とシークレットを控える。
5. PC で次を実行する（ブラウザでログインし直し、Kindle にログイン情報を送り直す）。

```
python kindlectl.py gdrive-setup --client-id 123456-xxxx.apps.googleusercontent.com --client-secret GOCSP
X-xxxx
```

一度指定すれば、以後の `gdrive-setup --reauth` でも同じクライアント ID を使う（`server\gdrive\rclone.conf` に保存される）。

13-4. 操作

場所	操作
KOReader	メニュー → ツール → **Google Drive 同期**（今すぐ同期 / 前回の結果 / 電源ボタンで復帰したときに同期 / 同期フォルダを開く）
KUAL	**Google Drive** → GDrive: Sync now / Sync on power button ON / Sync on power button OFF / Status
PC	`python kindlectl.py gdrive-sync` / `gdrive-status` / `gdrive-ls` / `gdrive-log` / `gdrive-wake on` / `gdrive-wake off`

- KOReader のファイルブラウザで `/mnt/us/gdrive` を開き、長押し → **ホームフォルダに設定**しておくとよい。
- 復帰時の同期を止める場合は、KOReader のメニューの「電源ボタンで復帰したときに同期」のチェックを外す。KUAL の **Sync on power button OFF**、PC の `gdrive-wake off` でも同じである。設定は `gdrive/config.sh` の `SYNC_ON_WAKE` に保存され、再起動後も維持される。
- 設定は `python kindlectl.py gdrive-set 項目 値` で変更できる。項目は付録A-3 を参照する。

---

14. 付録A 設定項目一覧

A-1. server\config.json（PC）

変更後は python serve.py restart で反映する。

項目	構築時の値	内容
location_name	東京	地名（現在のデザインでは表示しない）
latitude / longitude	35.6812 / 139.7671	天気を取得する地点
timezone	Asia/Tokyo	タイムゾーン
device	pw3	解像度。pw3 / voyage（1072×1448）、kt2（600×800）
port	8080	配信ポート
refresh_minutes	10	描き直す間隔（分）
weather_minutes	未記入（＝30）	天気を取得する間隔（分）
kindle.host	192.168.1.177	Kindle の IP アドレス
kindle.user / port / identity	root / 22 / ~/.ssh/kindle	SSH の接続情報
upload_pin	空	アップロード画面の PIN（空なら不要）

A-2. `extensions/signage/config.sh`（Kindle）

`python kindlectl.py set 項目 値` で変更できる。

項目	構築時の値	内容
`SERVER_URL`	http://192.168.1.134:8080	配信元。PC なし運用では `https://j105588.github.io/kindle_dashboard`
`SIGNAGE_SOURCE`	dashboard	`dashboard`＝時計・天気 / `slides`＝写真
`DASH_REFRESH_MIN`	10	背景を取り直す間隔（分）。60を割り切れる値にする
`DASH_REFRESH_OFFSET`	9	取り直す分のずれ（09・19・29…分）
`SLIDE_INTERVAL`	60	スライドショー1枚の表示秒数
`TZ_OFFSET_MIN`	540	日本時間との差（分）。時計が9時間ずれる場合は0
`NTP_SERVER` / `TIME_SYNC_MIN`	ntp.nict.jp / 60	時刻合わせのサーバーと間隔。空にすると合わせない
`GRAB_TOUCH`	1	サイネージ中にタッチを独占する
`TAP_EXIT`	3	終了するタップ回数（0＝タップで終了しない）
`FREEZE_UI`	1	タッチを独占できたときだけ裏の画面を一時停止する
`BATT_LOW_PCT`	15	この%以下で電池マークを表示する（充電中は出さない。0＝表示しない）
`BATT_CHG_ICON`	1	充電中は充電マークを表示する（0＝表示しない）
`WIFI_AUTO_ON`	1	Start 時に Wi-Fi を自動で ON にする
`WIFI_SAVE`	1	背景を取りに行くときだけ Wi-Fi をつなぐ（0＝つなぎっぱなし）。切れている間の `kindlectl.py`・スマホ送信は、次につながったときに送られる
`WIFI_LINGER_SEC`	60	背景を取り終えてから Wi-Fi を切るまでの秒数（この間に PC からの送信を受け付ける）
`WIFI_HOLD_MIN`	10	PC から操作されたら、最後の操作からこの分数は Wi-Fi を切らない（`she`

項目	構築時の値	内容
		ll でログイン中も切らない）
FRONTLIGHT_OFF	1	サイネージ中はフロントライトを消す（終了時に元の明るさへ戻す）
SS_SOURCE	images	スクリーンセーバーの内容（images / dashboard）
SS_ORDER	sequential	sequential＝順番 / random＝ランダム
SS_METHOD	auto	auto＝使えれば native / draw
SS_REFRESH_MIN	0	dashboard のとき、スリープ中に更新する間隔（分。0＝更新しない）

A-3. extensions/gdrive/config.sh（Kindle）

python kindlectl.py gdrive-set 項目 値 で変更できる。

項目	既定値	内容
GDRIVE_REMOTE	kgdrive:Kindle	同期元（rclone のリモート名:Drive のフォルダ）
LOCAL_DIR	/mnt/us/gdrive	同期先
FILE_TYPES	pdf	同期する拡張子（例 "pdf epub cbz"）
TRASH_DIR	/mnt/us/gdrive_trash	削除されたファイルの保管先
TRASH_DAYS	14	保管日数（0＝すぐ削除）
MAX_DELETE	20	1回の同期で削除してよい最大数
SYNC_ON_WAKE	1	電源ボタンで復帰したときに同期する（gdrive-wake on / off で切り替え）
MIN_GAP_MIN	3	前回の同期からこの分数以内の復帰では同期しない
WIFI_AUTO_ON	0	Wi-Fi が OFF のとき、一時的に ON にして同期する

15. 付録B ファイル配置

B-1. PC（C:\dev\tmp\kindle_dashboard）

```
.github\workflows\dashboard.yml   PC なし運用のワークフロー
docs\                             本書・プレビュー画像
kindle\                           Kindle に置くもの
  extensions\signage\             サイネージ・スクリーンセーバー
  extensions\gdrive\              Google Drive 同期
  koreader\plugins\gdrivesync.koplugin\   KOReader プラグイン
  signage\slides\ , screensaver\  画像の置き場所
server\
  config.json                     設定
  render.py / serve.py            描画・配信
  upload.py                       スマホ用アップロード画面
  convert_images.py               画像変換
  kindlectl.py / gdrive.py        SSH 操作・Drive 同期の設定
  setup_ssh.ps1                   SSH 鍵の登録
  requirements.txt
  out\                            描画結果・ログ（Git 対象外）
  gdrive\                         rclone 本体・Google のログイン情報（Git 対象外・公開禁止）
tools\touchgrab.c                 タッチ独占プログラムのソース
```

B-2. Kindle（/mnt/us = E:\）

```
documents\KUAL.sh , KUAL.jar      KUAL（PEKI）
extensions\signage\               サイネージ（config.sh / bin / img / menu.json）
extensions\gdrive\                Drive 同期（config.sh / bin / rclone.conf / cacert.pem / state.txt / gd
rive.log）
koreader\                         KOReader（plugins\gdrivesync.koplugin を含む）
mrpackages\                       MRPI のインストール待ちパッケージ
usbnet\etc\authorized_keys        SSH の公開鍵
signage\screensaver\ , slides\    画像
signage\signage.log               サイネージのログ
signage\.backup\                  update 時のバックアップ
gdrive\                           Drive から同期した PDF
gdrive_trash\                     Drive で削除されたファイルの保管先
update.bin.tmp.partial\           OTA 抑止用の空フォルダ
```

---

16. 付録C コマンド一覧

すべて `C:\dev\tmp\kindle_dashboard\server` で実行する。

C-1. serve.py

コマンド	内容
`python serve.py`	前面で起動する
`python serve.py --demo`	ダミーの天気で起動する
`python serve.py start` / `stop` / `restart` / `status` / `log`	バックグラウンド運用
`python serve.py autostart on` / `off`	自動起動の設定・解除

C-2. render.py

コマンド	内容
`python render.py --demo`	ダミーの天気で描画する
`python render.py --device kt2`	機種を指定して描画する
`python render.py --refresh-weather`	保存済みの天気が古ければ取り直して描画する（Actions用）

C-3. kindlectl.py

コマンド	内容
`check`	接続テストと状態表示
`push パス… [--to slides] [--replace] [--show]`	画像を変換して送る
`ls` / `rm 名前` / `clear --to 種類`	画像の一覧・削除
`start` / `stop`	サイネージの開始・終了
`ss-on` / `ss-off` / `preview`	スクリーンセーバーの操作
`set 項目 値` / `get [項目]`	signage の設定の変更・表示
`log [-n 行数]`	サイネージのログ
`update [--dry-run] [--with-config]` / `rollback`	プログラムの更新・取り消し
`shell`	Kindle にログインする
`gdrive-setup [--folder 名前] [--reauth] [--client-id ID --client-secret SECRET]`	Drive 同期の準備
`gdrive-sync` / `gdrive-status` / `gdrive-ls` / `gdrive-log`	Drive 同期の実行・確認
`gdrive-wake on` / `off`	電源ボタンで復帰したときの同期の切り替え
`gdrive-set 項目 値` / `gdrive-get [項目]`	gdrive の設定の変更・表示

Kindle が `WIFI_SAVE=1` で Wi-Fi を切っている間は、どのコマンドも次に Wi-Fi がつながるまで（最大 `DASH_REFRESH_MIN` 分）待ってから実行する。待つ上限は `config.json` の `"kindle": {"wait_min": 12}`（分）で変えられる（0＝待たない）。Ctrl+C で中止できる。

---

17. 付録D 障害対応

まず `python kindlectl.py log -n 20`（または `E:\signage\signage.log`）と `python serve.py log` を確認する。

D-1. 構築中に実際に発生した事象

事象	原因	対処
「Kindle をアップデート」がグレーアウトする	第7世代で Hotfix を更新として認識しない	Hotfix を `mrpackages` に入れて `;log mrpi` を実行する
サイネージの上に本や表紙が重なる	裏でホーム画面が描き直していた	裏の画面を一時停止する（`FREEZE_UI=1`）
タップが裏に届いて操作が乱れ、クラッシュする	タッチ入力を止めていなかった	touchgrab でタッチを独占する。固まったら電源を40秒長押しする
スクリーンセーバーが「自分→標準→自分」と切り替わる・ちらつく	Kindle が標準画像を遅れて描き直していた	`SS_METHOD=auto`（native 方式）にする
アップロード時に `code 127: Welcome to Kindle!`	SSH 接続時の PATH に `eips` がなかった	修正済み。古い `serve.py` が動いている場合は `python serve.py restart`
`天気の取得に失敗: HTTP Error 503` が毎時0分に出る	天気サービスが毎時0分に混雑する	天気は7分・37分ごろに取得し、失敗時は前回データで描く（修正済み）
`requirements.txt` が見つからない	`server` フォルダの外で実行していた	先に `cd server` する

D-2. 一般的な事象

事象	確認事項
KUAL に Signage が出ない	`extensions\signage\config.xml` があるか
KUAL で押しても何も起きない	`.sh` と `config.sh` が LF か
「Cannot reach …」と出る	サーバーの稼働、`SERVER_URL`、ファイアウォール、機内モード、2.4GHz の Wi-Fi か
左上に `offline` と出る	配信元に3回以上接続できていない。上と同じ確認を行う
時計が9時間ずれる	`TZ_OFFSET_MIN` を0にする
PC なし運用で更新されない	Actions の失敗・停止、cron-job.org の履歴が204か（401・403はトークン切れ）
SSH でパスワードを聞かれる	鍵が使われていない。USBNetwork を OFF → USB 接続 → `setup_ssh.ps1` を再実行
`Connection refused`	USBNetwork が OFF、または Wi-Fi 経由の SSH が無効
`Connection timed out`	IP の誤り、Kindle がスリープ中、別の Wi-Fi
`REMOTE HOST IDENTIFICATION HAS CHANGED`	`ssh-keygen -R 192.168.1.177` を実行してから再接続する
スクリーンセーバーが変わらない	Status で ON か（再起動で OFF に戻る）、`signage\screensaver` に画像があるか
KUAL を開くと真っ白	KUAL の版が機種に合っていない。PEKI を入れ直す

D-3. Google Drive 同期

事象	対処
`rclone not installed` / `not authorized`	`python kindlectl.py gdrive-setup` を実行する
`cannot read Drive: directory not found`	Drive に「Kindle」フォルダがあるか、`--folder` の名前を確認する
`stopped: N files would be deleted`	意図した削除なら `gdrive-set MAX_DELETE 100` で上限を上げて再同期する
`invalid_grant` などログインの期限切れ	`python kindlectl.py gdrive-setup --reauth`
`Quota exceeded` / `rateLimitExceeded`（403）	rclone 共有のクライアント ID の上限。13-4 の手順で自分のクライアント ID を作り、`gdrive-setup --client-id ... --client-secret ...`
rclone が起動しない（`exec format`、`kernel too old` など）	カーネルが古い。rclone は v1.69.0 のままにする（`server\gdrive.py` の `RCLONE_VERSION`）
証明書エラー（`x509`）	時計のずれを確認する。`gdrive-setup` で証明書を送り直す

18. 付録E 撤去手順

本システムは本体のシステム領域を書き換えないため、以下で元の状態に戻る。

1. KUAL で **Screensaver: OFF** と **Signage: Stop** を実行する。
2. `python kindlectl.py gdrive-wake off` を実行する（Drive 同期を導入した場合）。
3. Kindle を USB 接続し、次を削除する。
   - `E:\extensions\signage`、`E:\signage`
   - `E:\extensions\gdrive`、`E:\koreader\plugins\gdrivesync.koplugin`、`E:\gdrive`、`E:\gdrive_trash`
4. PC で `python serve.py stop` と `python serve.py autostart off` を実行する。
5. PC なし運用を行った場合は、cron-job.org のジョブを停止し、GitHub のトークンを Revoke する。
6. Google の許可は https://myaccount.google.com/connections で取り消す。
7. SSH が不要なら KUAL で USBNetwork を OFF にする。完全に削除する場合は、同じアーカイブの `uninstall` の `.bin` を `;log mrpi` で入れる。

19. 付録F 安全上の注意

- **脱獄の維持:** OTA アップデートが入った場合は、ライブラリの **Run Hotfix** を実行する。`update.bin.tmp.partial` フォルダは削除しない。
- **SSH の鍵:** SSH では Kindle に管理者（root）としてログインする。秘密鍵 `C:\Users\<ユーザー名>\.ssh\kindle` は他人に渡さない。
- **外部公開の禁止:** SSH と `serve.py` は自宅の Wi-Fi 内だけで使う。ルーターのポート開放は行わない。
- **公開リポジトリ:** PC なし運用ではリポジトリが Public になる。トークンと `server\gdrive\`（Google のログイン情報）は絶対にリポジトリに含めない（`.gitignore` 済み）。
- **電源:** 常時表示は USB 給電で行う。給電しない場合は、右上の電池マークを充電の合図とする。
- **復旧:** 電源ボタンの約40秒長押しで再起動すると、画面の一時停止、タッチの独占、スクリーンセーバーの差し替えはすべて解除される。

---

20. 付録G 参考資料

資料	URL
Kindle Modding Guide	https://kindlemodding.org/
Installing KUAL and MRPI	https://kindlemodding.org/jailbreaking/Legacy/post-jailbreak/installing-kual-mrpi/
Installing KOReader	https://kindlemodding.org/jailbreaking/post-jailbreak/koreader.html
Universal Hotfix	https://github.com/KindleModding/Hotfix/releases
PEKI	https://github.com/KindleTweaks/PEKI
Snapshots of NiLuJe's hacks（USBNetwork）	https://www.mobileread.com/forums/showthread.php?t=225030
MobileRead Wiki: USBNetwork	https://wiki.mobileread.com/wiki/USBNetwork
Wi-Fi 経由 SSH で締め出された事例と復旧	https://www.mobileread.com/forums/showthread.php?p=4381502
KOReader	https://github.com/koreader/koreader/releases
rclone v1.69.0	https://github.com/rclone/rclone/releases/tag/v1.69.0
Open-Meteo	https://open-meteo.com/
cron-job.org	https://cron-job.org/
kindle_dashboard	https://github.com/J105588/kindle_dashboard

21. 変更履歴

日付	内容
2026-10-11	サイネージ・スクリーンセーバー・SSH 運用を構築
2026-10-12	GitHub Actions による PC なし運用、時刻合わせ、電池表示を追加（PC 側で実装）
2026-10-12	Google Drive 同期（rclone）と KOReader プラグインを追加。本書を作成
2026-10-12	Drive 同期のタイミングを「電源ボタンでの復帰時＋手動」に変更（定期同期と KOReader 起動時の同期を廃止。旧ファイルは kindlectl 実行時に自動削除）